

① Define Artificial Intelligence. Explain the foundation of AI in detail.

→ Artificial Intelligence: AI is an area of computer science that emphasizes the creation of intelligent machines that work and react like humans.

Foundations of AI:

- 1) Mathematics: Formal representation and proofs algorithms; Propositional logic computation
Turing tried to characterize exactly which functions are computable - capable of being computed.
 - (un)decidability: Incompleteness theory showed that in any formal theory, there are true statements that are undecidable.
 - (in)tractability: A problem is called intractable, if the time required to solve instances of the problem grows exponentially with the size of the instances.
 - Probability: predicting the future.
- 2) Economics: Economics is a study of how people make choices that lead to preferred outcomes.
 - Decision theory: It combines probability theory with utility theory, provides a formal and complete framework for decision made under uncertainty.
- 3) Neuroscience: Neuroscience is the study of the nervous system, particularly brain. Brain consists of nerve cells or neurons, Neurons are considered as computational units.
- 4) Psychology: Cognitive psychology views the brain as an information processing device. Common view among psychologists that a cognitive theory should be like a computer program. i.e. It should describe a detailed information processing mechanism whereby some cognitive function might be implemented.

- ②
- ⑤ Computer engineering: For artificial intelligence to ~~succed~~ succeed, we need two things: intelligence and artifact. The computer has been artifact of choice.
- ⑥ Control theory and cybernetics: Modern control theory, especially the branch known as stochastic optimal control, has its goal the design of system that maximize an objective function over time. This roughly OBJECTIVE FUNCTION matches our view of AI: designing systems that behave optimally.
- ⑦ Computer Science Fundamentals: Algorithms and datastructures, efficient data manipulation and problem solving techniques.
- ⑧ ~~robotics~~ Linguistics: Important for natural language processing, enabling machines to understand and generate human language.

② Discuss the PEAS specification of Biometric Authentication System

→

1) Performance:

- a) Accuracy: Correctly identifying authorized users and rejecting unauthorized users
- b) False Acceptance Rate: Percentage of unauthorized users wrongly authenticated
- c) False Rejection Rate: Percentage of authorized users wrongly denied access.
- d) Response time: Speed of authentication process.

2) Environment:

- a) Physical environment: office buildings, secure facilities, mobile devices
- b) Lighting conditions: Bright, dim, or variable lighting
- c) Device Types: Integration with computers, door, smartphones

3) Actuators:

- a) User interface for enrollment and authentication
- b) Mechanism for granting / denying access.
- c) ~~Hardware~~ Hardware for accepting biometric inputs.

4) Sensors:

- a) Biometric input devices
- b) Fingerprint scanner for capturing unique patterns.
- c) Camera for facial recognition or iris scanning.
- d) Microphones for voice based authentication.

③ Give PEAS specification for Automated Taxi driver.

→ Agent Type: Taxi driver.

- 1) Performance : Safe, fast, legal, comfortable trip, maximize profits.
- 2) Environment: Roads, other traffic, pedestrians, customer.
- 3) Actuators: Steering, accelerator, brake, signal, horn, display.
- 4) Sensors: Camera, sonar, speedometer, GPS, odometer, accelerometer, engine sensor.

④ Give PEAS specification for Medical diagnosis system.

→ Agent Type: Medical diagnosis system.

- 1) Performance measure: Healthy patient, reduced costs
- 2) Environment: Patient, hospital, staff

4) Actuators : Display of questions, test, diagnoses, treatment.

5) Sensors: Keyboard entry of symptoms, findings, patient's answer.

⑤ Give PEAS specification of satellite image analysis system.

→ Agent type: Satellite image analysis system.

1) Performance: Correct image categorization.

2) Environment: Downlink from orbiting satellite

3) Actuators: Display of scene categorization

4) Sensors: Color pixel arrays.

Explain in own words (PEAS) if they ask for more marks.

⑥ Differentiate

1) Fully observable vs partially observable

2) Single agent vs multiagent.

3) Deterministic vs stochastic

4) Static vs dynamic

OR

Explain various types of environment in Artificial Intelligence.

→ 1) Fully observable vs partially observable:

a) A task environment is fully observable if the sensors detect the complete state of the environment, no need to maintain internal state to keep the track of environment.

b) A task environment may be partially observable, if noisy and inaccurate sensors parts of the state are not accessible for sensors.

c) A task environment may be even unobservable.

② Single agent vs multiagent.

a) Single agent: The agent operates independently without interaction or competition from other agent.

Eg: crossword puzzles, vacuum cleaner.

b) Multiagent: The agent operates in a setting where other agent exists, influencing outcomes through co-operation competition or other interactions.

Eg: chess, Taxi driving.

③ Deterministic vs stochastic

a) A task environment is deterministic if its next state is completely determined by its current state and by the action of agent.

Eg: crossword puzzle.

b) A task environment is stochastic if uncertainty about outcomes is quantified in terms of probabilities.

Ex: dice, poker game.

④ static vs dynamic:

a) A task environment is dynamic if it can change while the agent is choosing an action.

Ex: taxi driving.

b) A task environment is static if agent needs to keep looking at the world while deciding an action.

Ex: crossword puzzle.

① Define Artificial Intelligence? Explain Turing Test approach with example.

→ Artificial Intelligence is an area of computer science that emphasizes the creation of Intelligent ~~work~~ machine that work and react like humans.

The Turing Test approach

- 1) Turing Test is a method of inquiry in artificial intelligence for determining whether or not a computer is capable of thinking. This test was developed by Alan Turing ~~in~~ in 1950.
- 2) The Turing test is based on a party game called Imitation Game with some modifications. In this game, a human evaluator interacts with a computer and a human through a text based interface.
- 3) The evaluator asks questions to both the computer and human and tries to determine which one is computer.
- 4) If the evaluator cannot consistently distinguish between the computer and human then the computer is said to have passed the Turing test.

Ex: A human evaluator is communicating with two entities, one human and another computer through text based interface.

- 1) What is capital of France
- 2) What is meaning of life

The computer is programmed to respond in a way that mimics human conversation providing answers. The evaluator can also ask complex questions.

If evaluator cannot distinguish between the computer and human based on their responses, the computer is said to have passed the Turing test.

7

⑧ Define Agent . Explain any 3 types of Agent in Artificial Intelligence

→ Agent: An agent is any entity that can ~~perce~~ perceive its environment through sensors and act upon that environment through effectors.

Three types of agents:

1) Simple Reflex Agent.

These agents act based on simple condition-action rules. They do not maintain an internal state or model of the world.

Ex: A self driving car that automatically applies brake when obstacle is detected by the sensor.

Limitation: Limited to simple, reactive environments, cannot handle complex situation

2) Model based Reflex Agents:

These agents maintain an internal state or model of the world, allowing them to handle partially observable environment.

Ex: A thermostat that adjust temperature based on current readings and past temperature history.

These can handle complex situations.

3) Goal based agents:

These agents have a defined goal or objective and act to achieve it.

They consider current state of the environment and desired goal state.

Ex: A chess play AI that aims to win the game by making strategic moves

More flexible and adaptable than reflex agents.

- 4) Utility based agents: These agents consider not only the goal but also the desirability of different outcomes, maximizing their overall utility.
- 5) Learning Agents: These agents can improve their performance over time by learning from experience.

① Explain five components and well defined problem. Consider an 8 puzzle problem as an example.

-
- 1) The 8 puzzle consist of a 3X3 board with eight numbered tiles and a blank space. A tile adjacent to the blank space can slide into the space. The object is to reach a specified goal state.
 - 2) A typical instance of the 8 puzzle.

7	2	4
5		6
8	3	1

Start state

	1	2
3	4	5
6	7	8

Goal state.

- 3) Initial states: Any state can be designated as the initial state. Note that any given goal can ^{be} reached from exactly half of the possible initial states.
- 4) Actions: The simplest formulation defines the actions as movements of the blank space, Left, Right, up, down.
- 5) Transition model: Given a state and action, this returns the resulting state.
- 6) Goal test: This checks whether the state matches the goal configuration.
- 7) Path cost: Each step costs 1, so the path cost is the number of steps in path.

② In detail elaborate with neat diagram, the state space for the vacuum world. Links denotes L= Left R= Right S=Suck. ⑤

→ States: The state is determined by both the agent location and dirt location. The agent is one of two locations, each of which might or might not contain dirt. Thus there are $2 \times 2^2 = 8$ possible world states.

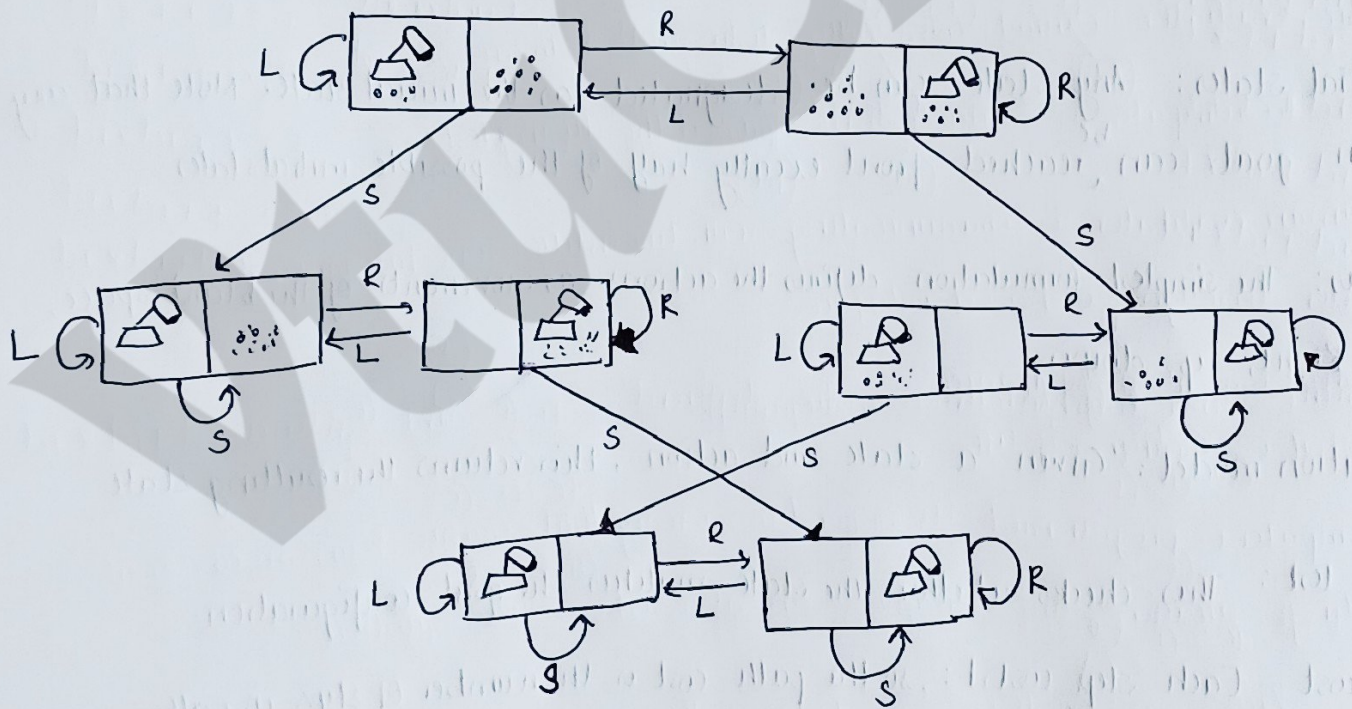
1) Initial state: Any state can be designated as initial state.

2) Actions: Each state has 3 actions: Left, Right and Suck.

3) Transition model: The actions have their expected effects, except that moving left in the leftmost square, moving right in the rightmost square and sucking in a clean square have no effect.

4) Goal test: Checks whether all the squares are clean.

5) Path cost: Each step costs 1, so the path cost is the number of steps in the path.



③ Explain Depth limited Search and Iterative Deeping DFS search with an example.

→ Depth Limited Search:

DLS imposes a maximum depth limit on the exploration of the search tree. This prevents the search from getting stuck in infinitely deep branches.

- Working
- 1) Start at root node
 - 2) Explore the children of the current node in a depth first manner.
 - 3) If a node's depth exceeds the predefined limit, stop exploring the branch
 - 4) If the goal node is found with depth limit, return success.
 - 5) If the depth limit is reached without finding the goal return failure.

Algorithm:

function Depth-limit-search (problem, limit) returns a solution, or failure/cutoff
return recursive-DLS (Make-Node (problem.INITIAL-STATE), problem, limit)

function RECURSIVE-DLS (node, problem, limit) returns a solution, or failure/cutoff
if problem.Goal-test (node.state) then return solution (node)

else if limit = 0 then return cutoff
else

cutoff-occured? $\leftarrow$ false

for each action in problem.ACTION (node.state) do

child $\leftarrow$ child-Node (problem, node, action)

result $\leftarrow$ Recursive-DLS (child, problem, limit-1)

if result = cutoff then cutoff-occured? $\leftarrow$ true

else if result $\neq$ failure then return result

if cutoff-occured? then return cutoff else return failure

Iterative deepening DFS

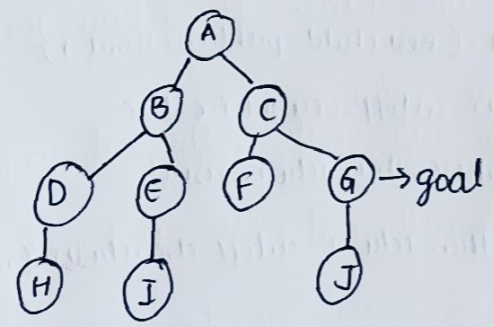
- 1) Iterative deepening is a general strategy often used in combination with depth-first tree search, that finds the best depth limit.
- 2) It does this by gradually increasing the limit - first 0, then 1, then 2 and so on until the goal is found
- 3) In an iterative deepening search, the nodes on the bottom level are generated once, those on the next to bottom level are generated twice, and so on upto the children of the root, which are generated d times.
- 4) The total number of nodes generated in worst case is $N(IDS) = (d)b + (d-1)b^2 + \dots + (1)b^d$.

Algorithm:

function iterative-deepening-search(problem) returns a solution, or failure
 for depth = 0 to ∞ do
 result $\leftarrow$ depth-limited-search(problem, depth)
 if result $\neq$ cutoff then return result.

Ex

Level 0
 Level 1
 Level 2
 Level 3



Iteration 0: A

Iteration 1: A $\rightarrow$ B $\rightarrow$ C

Iteration 2: A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E $\rightarrow$ C $\rightarrow$ F $\rightarrow$ G

④ Explain problem solving agent in detail with example.

⑤

→ Problem solving agent is a goal oriented agent that uses a systematic approach to find solutions to complex problems.

1) Problem Formulation

- Goal: The desired end state or objective
- Initial state: The starting point or current situation
- Actions: Possible moves or operations can be formed.
- Transition model: Describes how action change current state.
- Path cost: A function that evaluates the quality of a solution.

2) Search strategies:

- Uniformed Search: Algorithm that explore the search space without using any domain specific knowledge. Eg: BFS, DFS
 - Informed search: Algorithm that utilize heuristics or domain knowledge to guide the search process. Eg: A*, greedy best-first search.
- 3) Execution phase: Once a solution path is found, the agent executes the sequence of action to reach the goal state.
Eg: 8 puzzle problem.

Note: If this question asked for more than 6 mks, then explain 8 puzzle problem.

Application of problem solving agents

- Game playing: Chess, video games.
- Robotics: Path Planning.
- Logistics: Transportation.
- Artificial Intelligence Planning: Developing plans to achieve goals.

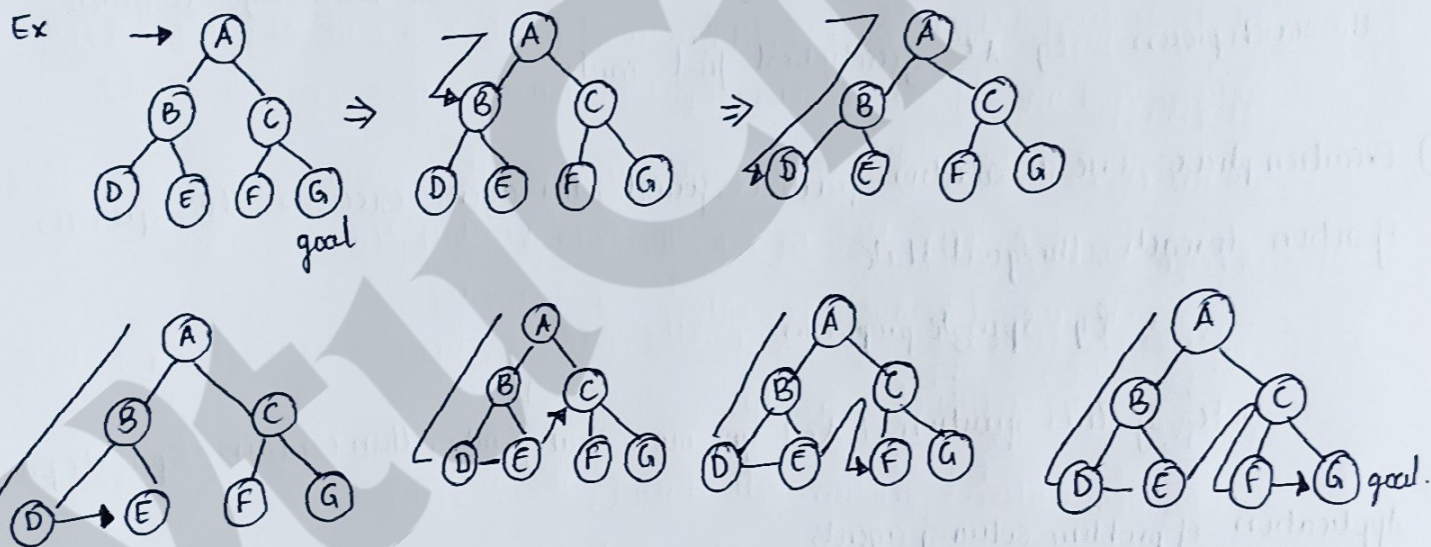
③ Explain how DFS and uniform cost search algorithm works in AI with suitable examples

→ DFS (Depth first search)

DFS explores a graph or tree by going as deep as possible along each branch before backtracking. It uses a stack to keep track of the nodes to explore.

How it works:

- 1) Start at the root node
- 2) Explore one of its children
- 3) Recursively explore that child's children and so on.
- 4) If dead end is reached, backtrack the parent node
- 5) Explore the next child of parent node.
- 6) Repeat steps 2-5 until the goal node is found.



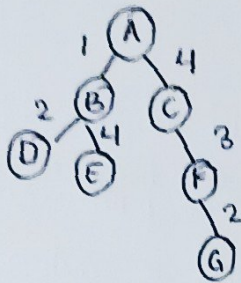
* Uniform cost search:

ucs finds the least cost path from the initial state to goal state by exploring nodes in order of their path cost. It uses priority queue to store nodes.

How it works.

- 1) start at the root node and initialize cost to 0
- 2) Expand the least cost node from the priority queue.
- 3) Add all unvisited neighbors to the queue with their cumulative costs.
- 4) Repeat until the goal node is expanded.

ex:

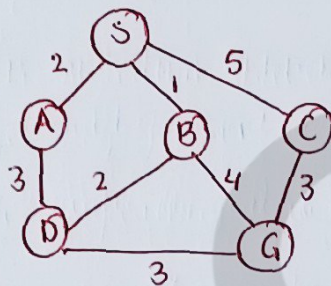


$A \rightarrow B = \text{cost} = 1$ $\rightarrow$ $B \rightarrow D = \text{cost} = 3$
 $\rightarrow$ $B \rightarrow E = \text{cost} = 5$

$A \rightarrow C = \text{cost} = 4$ $\rightarrow$ $C \rightarrow F = \text{cost} = 7$
 $\rightarrow$ $F \rightarrow G = \text{cost} = 9$

$A \rightarrow C \rightarrow F \rightarrow G: \text{cost} = 9 //$

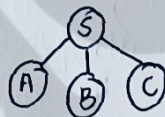
⑥ For a given graph below apply BFS and DFS search algorithm



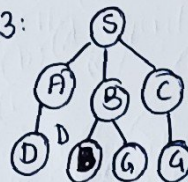
→ step 1:

⑤

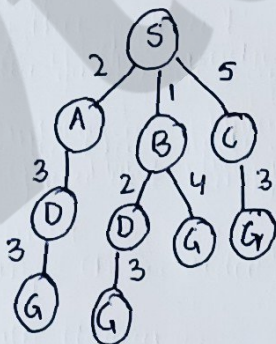
step 2:



step 3:



step 4:



level 0

level 1

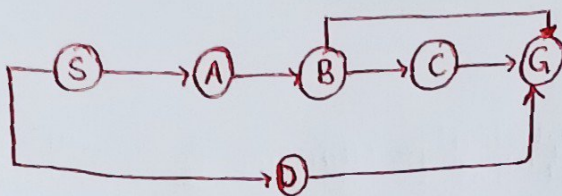
level 2

level 3

BFS: $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow D \rightarrow G$

DFS: $S \rightarrow A \rightarrow D \rightarrow G$

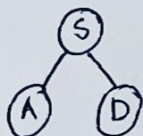
5



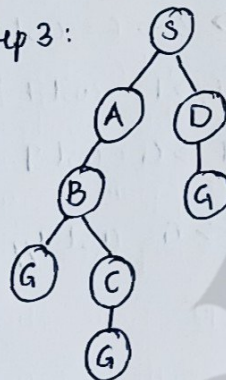
BFS

step 1: S

step 2:

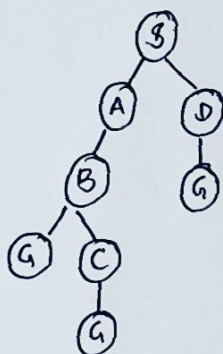


step 3:



Path: $S \rightarrow A \rightarrow D \rightarrow B \rightarrow G$

DFS



Path: $S \rightarrow A \rightarrow B \rightarrow G$

① What do you mean by knowledge based agents? Explain with an algorithm.

→ 1) Knowledge based agent are those agent who have the capability of maintaining internal state of knowledge, reason over that knowledge, update their knowledge after observation and take actions.

2) Knowledge based agents are composed of two ~~pos~~ main parts.

1) knowledge base

2) Inference system.

3) Knowledge base: It is a collection of ~~base~~ sentences. These sentences are expressed in a language which is called a knowledge representation language.

Inference system: Inference means deriving new sentences from old.

Inference system allows us to add a new sentence to the knowledge base.

4) Algorithm:

Following are two operations which are performed by KBA in order to show intelligent behaviour.

1) Tell: This operation tells the knowledge base what it perceives from the environment

2) Ask: This operation asks the knowledge base what action it should perform.

→ function KB-Agent(percept) returns an action

persistent: KB, a knowledge base

t, a counter, initially 0, indicating time

Tell(KB, Make-Percept-Sentence(percept, t))

action ← Ask(KB, Make-Action-Query(t))

Tell(KB, Make-Action-Sentence(action, t))

t ← t + 1

return action.

5) Each time when the function calls, it performs three operations

1) Firstly it TELLS the KB what it perceives.

2) Secondly, it ASKS KB what action it should take

3) Third agent program TELLS the KB that which action was chosen.

4) The MAKE-PERCEPT-SENTENCE generates a sentence as setting that the agent perceived the given percept at givertime.

5) The MAKE-ACTION-QUERY generates a sentence to ask which action should be done at current time.

6) The MAKE-ACTION-SENTENCE generates a sentence which asserts that the chosen action was executed.

⑤ Explain the wumpus world with example?

→ 1) The ~~w~~ wumpus world is a cave which has 4/4 rooms connected with passageways.

2) So there are 16 ~~rooms~~ rooms which are connected with each other.

3) We have a knowledge based agent who will go forward in this world.

4) The cave has a room with a beast which is called wumpus, who eats anyone who enters the room.

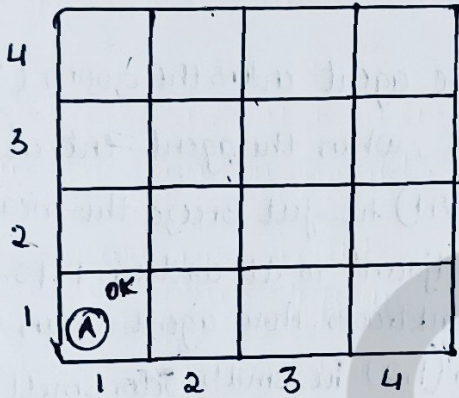
5) The wumpus can be shot by the agent, but the agent has single arrow.

4	 stench		 breeze	PIT
3		 stench Gold	PIT	 breeze
2	 stench		 breeze	
1	 skull	 breeze	PIT	 breeze
	1	2	3	4

Conditions:

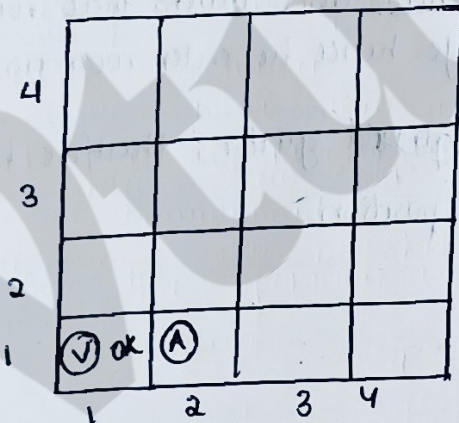
- 1) The room adjacent to wumpus are stench (odor smell)
 - 2) The room adjacent to pit has breeze
 - 3) There will be glitter in the room if and only if the room has gold.
 - 4) Actuators: left move, right move, grab, release
- sensors: stench, breeze, glitter.

Agent 1st step:



The current room (1,1) is safe.

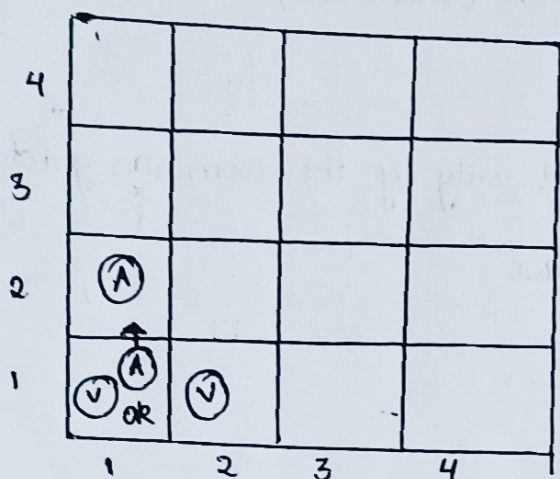
Agent 2nd step



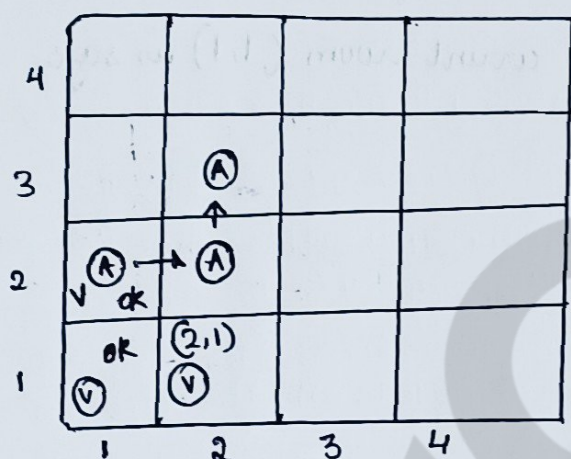
The current room has breeze, this means that the room having breeze has adjacent room containing pits. So the agent backtracks.

Agent 3rd step.

(4)



Now the agent enters room (1,2). The agent ~~sense~~ ~~sense~~ sense odor smell, this means the room containing odor smell has adjacent room with containing wumpus.



Now the agent enters the room (2,2) because, when the agent entered the room (2,1) he felt Breeze this means the rooms adjacent to it contain PITS. So the agent backtracks. Now agent when enters the room (1,2) he smells odor smell, and also thinks that room (3,1) and (2,2) has pits, but when he enters the room no (1,2) he smell odor and thinks that room no (2,2) is safe. hence he enter room no (2,2)

When he enters room no (2,2) he finds ~~gold~~ glitter, therefore he grabs the gold and comes back to initial position.

③ What is propositional logic? Explain its connectiveness with respect to truth table.

→ Propositional logic is a branch of logic that deals with propositions and their relationship. A proposition is a declarative statement that is either true or false but not both.

There are five connectiveness in common use.

1) $\neg$ (not) : A sentence such as $\neg w_{1,3}$ is called the negation of $w_{1,3}$.

Negation	
P	$\neg P$
T	F
F	T

2) $\wedge$ (and) : A sentence whose main connective is $\wedge$, such as $w_{1,3} \wedge p_{3,1}$ is conjunction

Conjunction		
P	q	$P \wedge q$
T	T	T
T	F	F
F	T	F
F	F	F

3) $\vee$ (or) : A sentence using $\vee$, such as $(w_{1,3} \wedge p_{3,1}) \vee w_{2,2}$ is a disjunction

P	q	$P \vee q$
T	T	T
T	F	T
F	T	T
F	F	F

4) $\Rightarrow$ (Implication): A sentence such as $(w_{1,3} \wedge p_{3,1}) \Rightarrow \neg w_{2,2}$ is called implication

Implication

p	q	$q \Rightarrow p$
T	T	T
T	F	F
F	T	T
F	F	T

5) $\Leftrightarrow$ (if and only if): The sentence $w_{1,3} \Leftrightarrow \neg w_{2,2}$ is biconditional.

If and only if

p	q	$p \Leftrightarrow q$
T	T	T
T	F	F
F	T	F
F	F	T

④ What is heuristic function? Explain A* search strategy in detail.

→

② Heuristic function is a function used in search and optimization algorithms to estimate the cost or value of a solution relative to a goal.

A* search algorithm

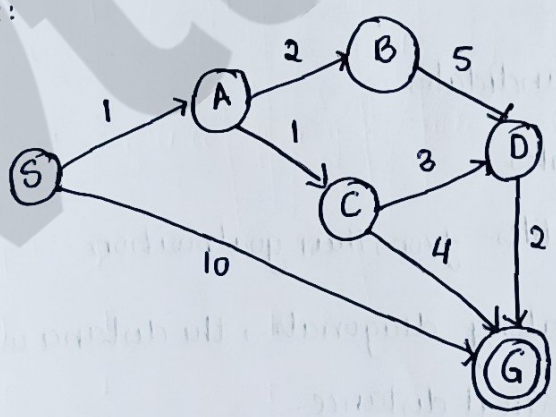
- 1) A* search algorithm find the shortest path through the search space using heuristic function.
- 2) It uses $h(n)$ and cost to reach the node 'n' from the start stage $g(n)$
- 3) This algorithm expands less search tree and provides optimal results faster.
- 4) It is similar to ~~ucs~~ UCS except that it uses $g(n)+h(n)$ instead of $g(n)$.
- 5) $F(n) = g(n) + h(n)$

$f(n)$ = estimated cost of cheapest solution

$g(n)$ = Cost to reach node "n" from start state

$h(n)$ = Cost to reach from node "n" to goal node

Example:



State	$h(n)$
S	5
A	3
B	4
C	2
D	6
G	0

$$① \quad S \rightarrow A = F(n) = 1 + 3 = 4$$

$$S \rightarrow G = F(n) = 10 + 10 = 10$$

$$② \quad S \rightarrow A \rightarrow B = F(n) = 3 + 4 = 7$$

$$S \rightarrow A \rightarrow C = F(n) = 2 + 2 = 4 \quad \checkmark$$

$$③ \quad \cancel{S \rightarrow A \rightarrow B \rightarrow D \rightarrow E \rightarrow F}:$$

$$S \rightarrow A \rightarrow C \rightarrow D = F(n) = 5 + 6 = 11$$

$$S \rightarrow A \rightarrow C \rightarrow G = F(n) = 6 + 0 = 6$$

$$S \rightarrow A \rightarrow C \rightarrow \textcircled{G} \quad \text{cost: } 6$$

⑤ Explain Heuristic function w.r.t to 8-puzzles.

→ 1) The average cost of a randomly generated 8 puzzle instance is about 22 steps.

2) The branching factor is about 3. When the empty tile is in the middle four moves are possible, when it is in corner two moves are possible, when it is along an edge three moves are available.

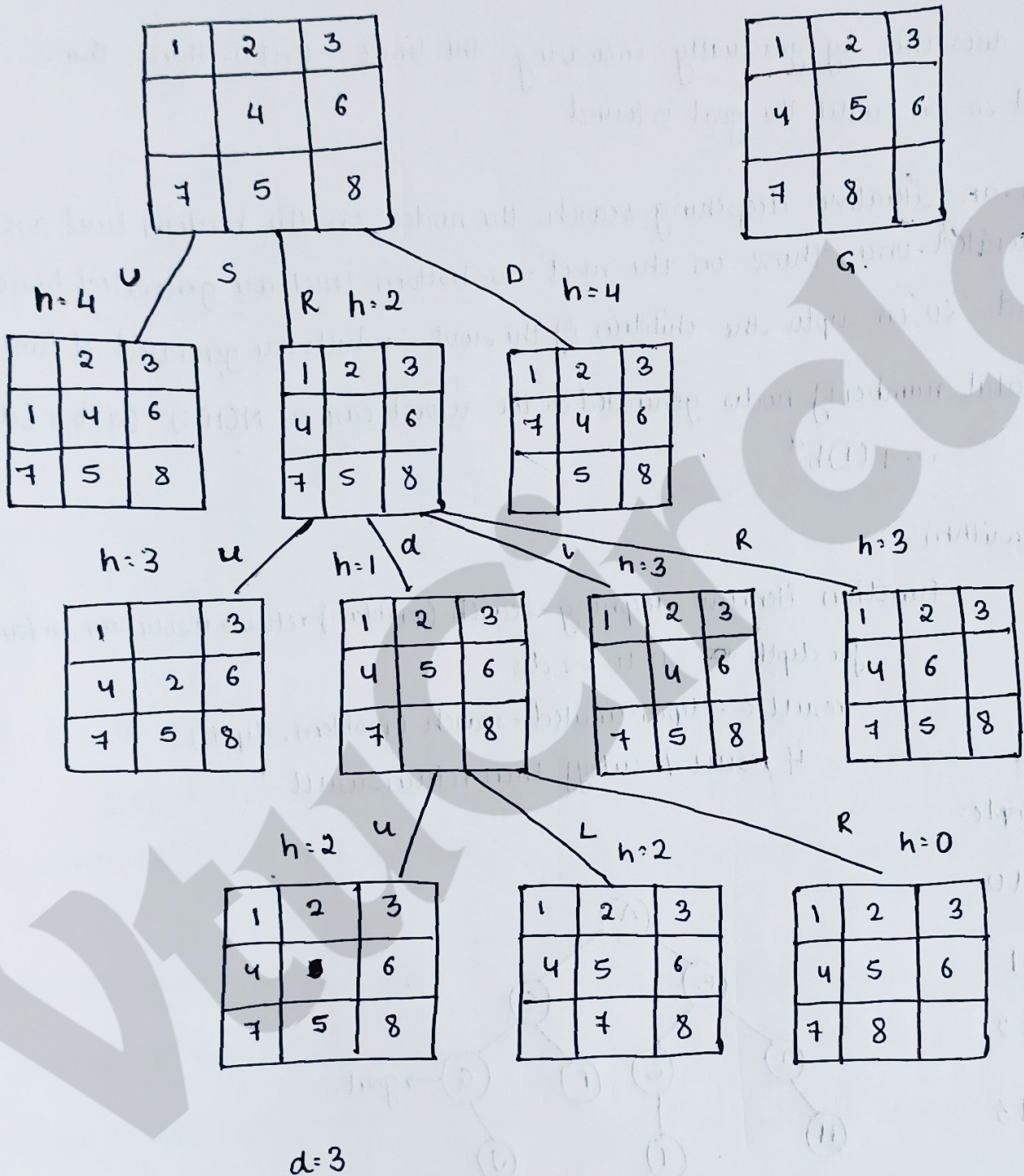
3) This means that an exhaustive tree search to depth 22 would look at about $3^{22} \approx 3.1 \times 10^{10}$

4) Here are commonly two used candidates.

h_1 = the number of misplaced tiles

h_2 = the sum of distances of the tiles from their goal position.

5) Because the tiles cannot move along diagonals, the distance w.r.t count is sum of horizontal and vertical distance.



⑥ Explain iterative deepening DFS in detail.

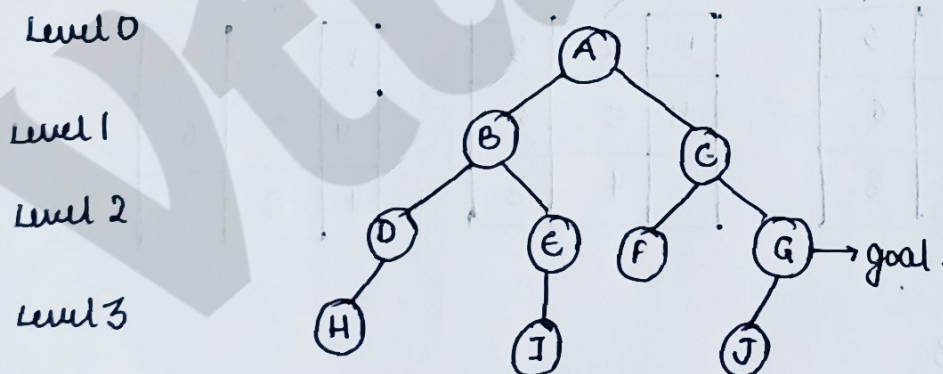
(10)

- 1) Iterative deepening is a general strategy often used in combination with depth-first tree search, that finds the best depth limit.
- 2) It does this by gradually increasing the limit - first 0, then 1, then 2 and so on until the goal is found.
- 3) In an iterative deepening search, the nodes on the bottom level are generated once, those on the next-to-bottom level are generated twice and so on upto the children of the root, which are generated d times.
- 4) Total number of nodes generated in the worst case is $N(IDS) = (d)b + (d-1)b^2 + \dots + (1)b^d$.

Algorithm:

```
function iterative-deepening-search (problem) returns a solution, or failure.  
  for depth  $\infty = 0$  to  $\infty$  do  
    result  $\leftarrow$  depth-limited-search (problem, depth)  
    if result  $\neq$  cutoff then return result.
```

Example:



Iteration 0: A

Iteration 1: A $\rightarrow$ B $\rightarrow$ C

Iteration 2: A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E $\rightarrow$ C $\rightarrow$ F $\rightarrow$ G.

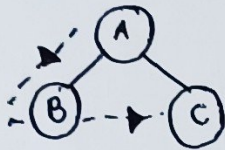
Tree structure

Level 0:



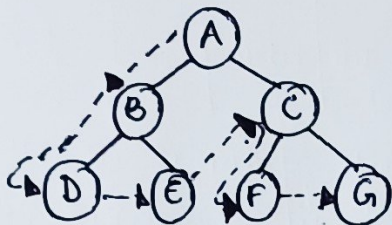
A

Level 1:



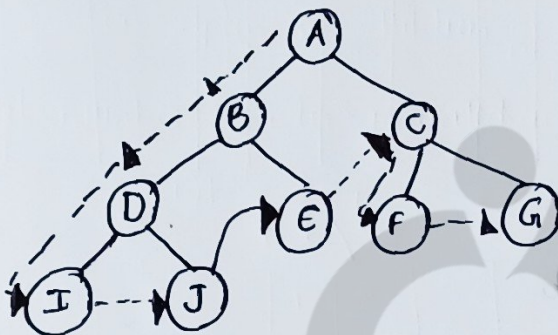
$A \rightarrow B \rightarrow C$

Level 2:



$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$

Level 3



$A \rightarrow B \rightarrow D \rightarrow I \rightarrow J \rightarrow E \rightarrow C \rightarrow F \rightarrow G$

⑦ List the difference between BFS and DFS

BFS

DFS

→ 1) Explores neighbours level by level

Explores as deep as possible along a branch before backtracking.

2) Data structure used is Queue

Data structure used is stack.

3) Level order traversal

Depth first traversal

4) Space complexity is higher

Space Complexity is lower.

5) Time complexity is $O(V+E)$

Time complexity is $O(V+E)$

6) Detects cycle in undirected graph

Detects cycle in directed graph.

7) Requires more memory for wide graphs

Requires less memory compared to BFS

8) Represent the following sentences in reasoning patterns in propositional logic. (12)

- 1) I am not attending class
- 2) I am ~~not~~ attending class and lab.
- 3) I am attending class or lab.
- 4) If I attend the class then I am attending lab.
- 5) I am not attending class and lab.
- 6) I am attending class only.

→ Let (C) : I am attending class.
 (L) : I am attending lab.

Representations.

- 1) I am not attending class.

$[\neg C]$ or $[\neg C]$

- 2) I am ~~not~~ attending class and lab

$[C \wedge L]$ or $[C \text{ and } L]$

- 3) I am attending class or lab

$[C \vee L]$ or $[C \text{ or } L]$

- 4) If I attend the class then I am attending lab.

$[C \Rightarrow L]$ or $[C \text{ implication } L]$

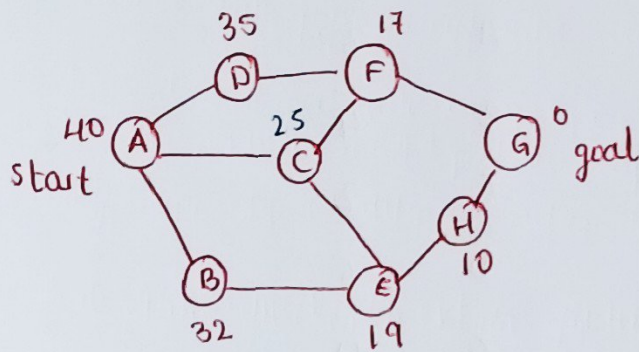
- 5) I am not attending class and lab

~~$[\neg(C \wedge L)]$~~ $[\neg(C \wedge L)]$

- 6) I am attending class only.

$[C \wedge \neg L]$

- Q In the below graph, find the path from A to G using greedy Best first search.
- The values in the table represent heuristic values of reaching the goal node in pass current node.

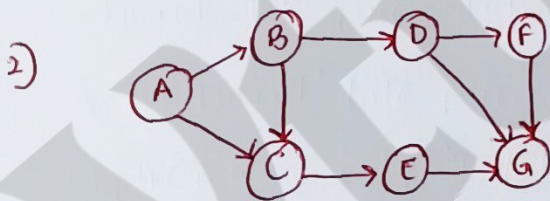


→ 1) $f(D) = h(D) = 35$
 $f(C) = h(C) = 25$ ✓
 $f(B) = h(B) = 32$

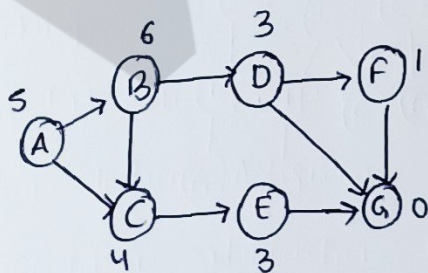
2) $f(F) = h(F) = 17$ ✓
 $f(E) = h(E) = 19$

3) $f(G) = h(G) = 0$ ✓
 $f(D) = h(D) = 35$

Path: $A \rightarrow C \rightarrow F \rightarrow G$



A	5
B	6
C	4
D	3
E	3
F	1
G	0



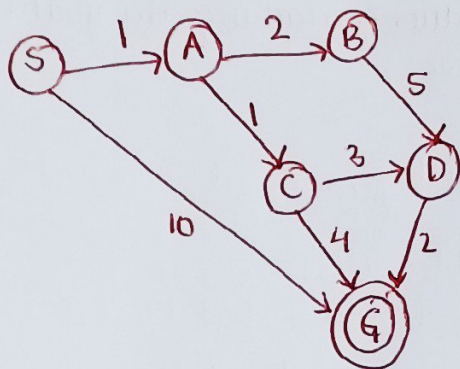
1) $f(B) = h(B) = 6$
 $f(C) = h(C) = 4$ ✓

2) $f(E) = h(E) = 3$ ✓

3) $f(G) = h(G) = 0$

Path: $A \rightarrow C \rightarrow E \rightarrow G$

10. In below graph, find the path of A to G using A* search algorithm.



State	$h(n)$
S	5
A	3
B	4
C	2
D	6
G	0

→ 1) $S \rightarrow A = f(n) = 1 + 3 = 4 \checkmark$

$S \rightarrow G = f(n) = 10 + 0 = 10$

2) $S \rightarrow A \rightarrow B = f(n) = 1 + 3 + 4 = 8$

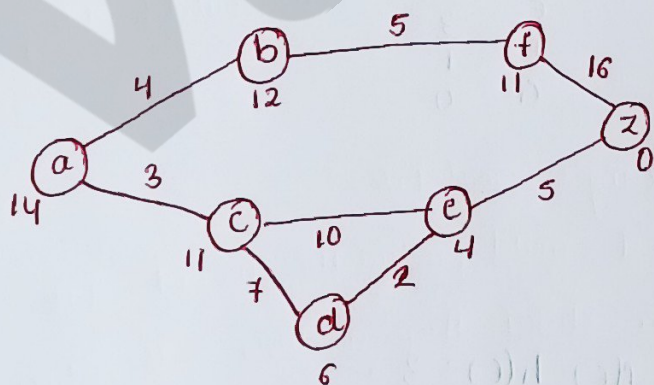
$S \rightarrow A \rightarrow C = f(n) = 2 + 2 = 4 \checkmark$

3) ~~$S \rightarrow A \rightarrow C = f(n) =$~~
 ~~$S \rightarrow A \rightarrow G = f(n) =$~~

3) $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D = f(n) = 5 + 6 = 11$

$S \rightarrow A \rightarrow C \rightarrow G = f(n) = 6 + 0 = 6 \checkmark$

path: $S \rightarrow A \rightarrow C \rightarrow G$: Total cost: 6



→ State $h(n)$

a 14

b 12

c 11

d 6

e 4

f 11

z 0

$$1) a \rightarrow b : F(n) : 4 + 12 = 16$$

$$a \rightarrow c : F(n) : 3 + 11 = 14 \checkmark$$

$$2) a \rightarrow c \rightarrow e : F(n) : 13 + 4 = 17$$

$$a \rightarrow c \rightarrow d : F(n) : 10 + 6 = 16 \checkmark$$

$$3) a \rightarrow c \rightarrow d \rightarrow e : F(n) : 12 + 4 = 16$$

$$4) a \rightarrow c \rightarrow d \rightarrow e \rightarrow z : F(n) : 17 + 0 = 17$$

Path: $a \rightarrow c \rightarrow d \rightarrow e \rightarrow z$

cost: 17

Artificial Intelligence

- ① Explain syntax and semantics of first order logic with suitable examples and list drawbacks of propositional logic.

→ Syntax of first order logic:

Sentence $\rightarrow$ Atomic Sentence | Complex Sentence

Atomic sentence $\rightarrow$ Predicate | Predicate (Term ..) | Term = Term

Complex sentence $\rightarrow$ (Sentence) | [Sentence]

$\rightarrow \neg$ Sentence

$\rightarrow$ Sentence $\wedge$ Sentence

$\rightarrow$ Sentence $\vee$ Sentence

$\rightarrow$ Sentence $\Rightarrow$ Sentence

$\rightarrow$ Sentence $\Leftrightarrow$ Sentence

$\rightarrow$ Quantifier Variable, ... | Sentence

Term $\rightarrow$ Function(Term)

$\rightarrow$ Constant

$\rightarrow$ Variable

Quantifier $\rightarrow \forall / \exists$

Constant $\rightarrow A | x_1 | John | \dots$

Variable $\rightarrow a | x | s | \dots$

Predicate $\rightarrow$ True | False | After | loves | Raining | ...

Function $\rightarrow$ Mother | Leftleg | ...

operator precedence: $\neg, =, \wedge, \vee, \Rightarrow, \Leftrightarrow$

- 1) Models for first order logic: The models of a logical language are the formal structures that constitute the possible worlds under consideration.
- 2) Symbols: Symbols are the basic syntactic elements of first order logic. Symbols stand for objects, relations and functions.
- 3) Term: A term is a logical expression that refers to an object. Constant symbols are terms.
- 4) Atomic sentences: An atomic sentence is formed from a predicate symbol followed by a parenthesized list of terms. Eg: Brother (Richard, John)
- 5) Complex sentences: Complex sentences can be constructed using logical connectives just as in propositional calculus. Eg: $\neg \text{King}(\text{Richard}) \Rightarrow \text{King}(\text{John})$
- 6) Quantifiers express properties of entire collection of objects; instead of enumerating the object by name.

Drawbacks of Propositional logic

- 1) Propositional logic lacks the ~~ispe~~ expressive power to concisely describe an environment with many objects.
- 2) For example, we were forced to write a separate rule about breezes and pits for each square, such as $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
- 3) In English, it seems easy enough to say "Squares adjacent to pit are breezy"
- 4) The syntax and semantics of English somehow make it possible to describe the environment concisely.

② Define Universal and Existential instantiation and give examples for both.

Prove the following using forward chaining: "As per law", it is a crime for an American to sell weapon to hostile nations. Country E, an enemy of America has some missiles, and all the missiles were sold to it by ~~Stee~~ solar, who is an American citizen. Prove that "~~Stee~~ solar is a criminal".

→ Universal Instantiation: The rule says that we can infer any sentence obtained by substituting a ground term for the variable. Let $SUBST(\theta)$ denote the result of applying the substitution θ to sentences a . Then rule is written as

$$\frac{\forall v a}{SUBST(\{v/g\}, a)}$$

$$\text{Ex: } \forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$$

$$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John}).$$

Existential Instantiation: The existential sentence says there is some object satisfying a condition and the instantiation process is just giving a name to that object that name must not already belong to another object.

$$\frac{\exists v a}{SUBST(\{v/k\}, a)}$$

$$\text{Ex: } \exists x \text{ Crown}(x) \wedge \text{onHead}(x, \text{John})$$

$$\text{Crown}(c_1) \wedge \text{onHead}(c_1, \text{John})$$

Given Facts:

- 1) It is a crime for an American to sell weapons to hostile nations.
- 2) Country E is an enemy of America
- 3) Country E has missiles.
- 4) All missiles in Country E were sold to it by Solan.
- 5) Solan is an American citizen.

Steps to Prove

Step 1: From Fact 2: Country E is an enemy of America

Enemy of America (E) $\Rightarrow$ Hostile (E) : Thus Hostile (E) is true

Step 2: From Fact 3 and 4: Country E has missiles, and all missiles were sold to it by Solan

Has Missiles (E) $\wedge$ sold by (Solan, E) $\Rightarrow$ Sells weapons (Solan, E) is true

Step 3: From Fact 5: Solan is an American citizen : Thus American (Solan) is true

Step 4: American (Solan)

Sells weapons (Solan, E)

Hostile (E)

Thus Criminal (Solan) is true //

③ Explain the following with respect to first order logic.

1) Assertion and queries in first order logic

2) The kinship domain

3) Numbers, sets and lists.

→ Assertion:

Sentences are added to a knowledge base using TELL, exactly as in propositional logic. Such sentences are called as assertions.

Ex: John is a king, $TELL(KB, King(John))$, Richard is a person. $TELL(KB, Person(Richard))$. All kings are persons: $TELL(KB, \forall x King(x) \Rightarrow Person(x))$.

Asking Queries:

We can ask questions of the knowledge base using ASK. Questions asked with ASK are called queries or goals.

Ex: $ASK(KB, King(John))$ returns true. Any query that is logically entailed by the knowledge base should be answered affirmatively.

2) The kinship domain:

The objects in kinship domain are people. Male and Female.

Kinship relations: parenthood, brotherhood, marriage and so on, are represented by binary predicates Parent, sibling, Brother, sister, child, Daughter, son, wife etc.

We use functions for Mother and Father, because every person has exactly one of each of these.

Ex: One's mother is one's female parent: $\forall m, c \text{ Mother}(c) = m \Leftrightarrow \text{Female}(m) \wedge \text{Parent}(m, c)$

One's husband is one's male spouse: $\forall w, h \text{ Husband}(h, w) \Leftrightarrow \text{Male}(h) \wedge \text{Spouse}(h, w)$

Male and female are disjoint categories: $\forall x. \text{Male}(x) \Leftrightarrow \neg \text{Female}(x)$

3) Number, Sets and Lists:

Numbers: Numbers can be represented as objects in FOL, with function and predicates defining relationship

Example: $\text{Number}(0), \text{Number}(1) \dots \rightarrow$ Declares numbers as object.

$\text{Successor}(x, y) \rightarrow y$ is the successor of x

Arithmetic operations:

$\text{Add}(x, y, z) \rightarrow z = x + y$

$\text{Multiply}(x, y, z) \rightarrow z = x \times y$

Sets: Sets can be defined using predicates and membership relationship

Ex: $\text{Set}(s) = s$ is a set

$\text{Member}(x, s) \rightarrow x$ is an element of sets

$\text{Subset}(A, B) \rightarrow A$ is a subset of B

List: Lists are ordered sequence of elements. Represented using Nil for an empty list

$\text{cons}(h, t)$ for constructing a list with head h and tail t

Ex: List $[1, 2, 3]$:

$\text{cons}(1, \text{cons}(2, \text{cons}(3, \text{Nil})))$.

④ considering the digital circuit example, illustrate the seven steps in knowledge engineering process.

$\rightarrow$ 1) Identify the task! Define the problem that the knowledge based system needs to solve.

Ex: Design a knowledge based system to verify the correctness of a digital circuit design.

2) Assemble the Relevant knowledge : Gather the required knowledge for solving the problem.

Ex: Collect knowledge about , Basic components, Rules for Boolean algebra, Standard circuit design practices.

3) Choose a Representation: Select a representation to model the knowledge effectively

Ex: Use FOL to represent components and relationship.

Predicates :

$\text{Gate}(G, \text{Type}) \rightarrow G$ is a gate of type Type (AND, OR, NOR etc)

$\text{Input}(G, X) \rightarrow X$ is an input to gate G

$\text{Output}(G, Y) \rightarrow Y$ is an output to gate G .

4) Encode the knowledge : Implement the gathered knowledge in a computable form.

Ex: encoding : $\text{Gate}(G1, \text{AND})$

$\text{Input}(G1, A)$

$\text{Input}(G1, B)$

$\text{Output}(G1, Z)$

5) Test the knowledge Base : verify that the encoded knowledge base work correctly

Eg: Test scenarios

Input : $A=1, B=0$ for $G1$

output $Z=0$

6) Integrate the System: Combine the knowledge base with an inference engine and interface

Eg: Integrate : A reasoning engine to deduce circuit behaviour.

7) Maintain the System : update the system to incorporate new knowledge or adapt to changing requirements.

Ex: update knowledge base for new technologies (eg. quantum circuit)

⑤ Explain the steps involved in knowledge engineering with example.

→

1) Identify the Task: Define the purpose and scope of the knowledge system.

Ex: Create a system to assist doctors in diagnosing diseases based on symptoms.

2) Assemble the Relevant knowledge: Collect all necessary knowledge from domain, experts, textbooks, research, papers or databases.

Ex: ~~Collect all necessary knowledge from domain expert, textbooks, research, papers or data~~

Ex: Gather knowledge about symptoms and their relationships to diseases.

3) Choose a Representation: Decide how to model the knowledge for computational reasoning.

Ex: IF (Fever AND Cough AND Fatigue) THEN Diagnosis = Flu

4) Encode the knowledge: Convert the collected knowledge into a format that the computer system can process.

Eg: IF Chestpain AND Difficulty Breathing THEN Diagnosis = Pneumonia.

5) Test the knowledge Base: Verify the correctness and completeness of the knowledge base

Eg: Input: Fever, Cough, Fatigue.
Output: Flu

6) Integrate the system: Combine the knowledge base with an interface engine and user interface to create a functional system.

Eg: Use logical reasoning to deduce diagnoses based on input symptoms.

7) Maintain the System: Update and improve the system as knowledge evolves or new requirements emerge

Eg: Add new disease or update symptoms.

⑥ Explain forward chaining with example.

→ Forward chaining is a method of reasoning in Artificial Intelligence where inference starts with known facts and applies rules to extract more data until goal or conclusion is reached.

Working

- 1) Start with facts: Begin with the initial set of facts or knowledge
- 2) Apply rules: Check which rules can be triggered based on these facts
- 3) Infer new facts: Apply the rules to infer new facts
- 4) Repeat until goal: Continue applying rules until the desired conclusion is reached or no more rules can be applied.

Example: Diagnosing Fever

* Facts

- 1) Symptoms: High temperature
- 2) Symptoms: Headache

* Rules:

- 1) If the patient has a high temperature and headache, then the patient may have a fever.
- 2) If the patient has a fever and a rash, then the patient may have measles.

* Process

- 1) Known facts: High temperature, headache
- 2) Rule 1 triggered. = Fever.
- 3) No new rules can be triggered: Inference stops.

* Result: The system concludes that the patient may have a fever.

Application: Expert System, Business Rule Engines, Automation.

(7) Explain unification in Artificial Intelligence with example.

→ Unification is a process in AI that determines whether two logical expressions can be made identical by substituting variables with appropriate constants or other variables.

- 1) Substitution: Replacing a variable with a constant, another variable or term.
- 2) Most General Unifier: The simplest substitution that makes two expressions identical.

Steps of unification

- 1) Compare the structure of the two expressions.
- 2) If a mismatch is found replace variables with constant or terms, to resolve the mismatch.
- 3) Continue until the expressions are identical or unification fails.

Ex: $P(xy)$
 $P(az)$

unify x with a
unify y with z

$$P(xy) = P(az)$$

unification fails when the structure of the expressions differ $P(x)$ vs $Q(x)$
A variable cannot be consistently substituted $x = a$ & $x = b$, $a \neq b$.

Applications: Logic programming, Automated theorem, Natural language.

Module 5

① Explain backward chaining algorithm, with an example.

→

- 1) This algorithm work backward from the goal, chaining through rules to find known facts that support the proof.
- 2) It is called with the list of goal containing the original query and returns the set of all substitution satisfying query.
- 3) The algorithm takes the first goal in the list and finds every clause in the knowledge base whose head, unifies with the goal.
- 4) Each such clause creates a new recursive call in which body of the clause is added to the goal stack.

Algorithm:

function FOL-BC-ASK(KB, agent) returns a generator of substitution

return FOL-BC-OR(KB, query, {})

generator FOL-BC-OR(KB, goal, θ) yields a substitution

for each rule (lhs $\Rightarrow$ rhs) in Fetch-Rules-For-Goal(KB, goal) do

(lhs, rhs) $\leftarrow$ standardise variables ((lhs, rhs))

foreach θ' in FOL-BC-AND(KB, lhs, UNIFY(rhs, goal, θ)) do

yield θ'

generator FOL-BC-AND(KB, goals, θ) yields a substitution

if θ = failure then return

else if Length(goals) = 0 then yield θ

else do

first, rest $\leftarrow$ first(goals), rest(goals)

foreach θ' in FOL-BC-OR(KB, SUBST(θ , first), θ') do

foreach θ'' in FOL-BC-AND(KB, rest, θ') do

yield θ''

Ex: Diagnosing a fever caused by flu

Rule 1: Flu $\Rightarrow$ Fever

Rule 2: Flu $\Rightarrow$ Cough

Rule 3: Cough $\wedge$ Fever $\Rightarrow$ Flu

Fact 1: Cough = True

Fact 2: Fever = True.

Goal: person has flu

BC Execution

1) Start with goal: Prove Flu

2) Find Rule: Rule 3: Cough $\wedge$ Fever $\Rightarrow$ Flu

3) Subgoals: Cough = True, Fever = True

4) All subgoals proven $\therefore$ Flu = True.

② Define classical planning, with the blocks world example explain the same in detail.

→ Classical planning is a branch of AI that focuses on planning sequences of action to achieve a desired goal. It uses a factored representation of the world where states are described as a set of variables, and it relies on logical reasoning to find the solution.

Blocks world:

The blocks world is a classical planning domain involving blocks that can be stacked on top of each other or placed on a table.

1) Objects: Block and table

Block A, Block B, Block C, Table.

2) states: On (Block A, Block B), On (Block B, Table), clear (Block A)

clear(X): Block X has no block on top.

3) Actions : $\text{PickUp}(\text{Block})$; $\text{PutDown}(\text{Block})$, $\text{stack}(\text{Block X}, \text{Block Y})$; $\text{unstack}(\text{Block X}, \text{Block Y})$

* Initial state: $\text{On}(\text{Block A}, \text{Block B})$, $\text{On}(\text{Block B}, \text{Table})$, $\text{On}(\text{Block C}, \text{Table})$, $\text{clear}(\text{Block A})$, $\text{clear}(\text{Block C})$.

* Goal state: $\text{On}(\text{Block B}, \text{Block C})$, $\text{On}(\text{Block A}, \text{Block B})$.

③ Write appropriate quantifiers for the following

- i) some students read well
- ii) some students like some books
- iii) some students like all books.
- iv) All student like some books
- v) All student like no books.

→ i) $\exists x (\text{student}(x) \wedge \text{ReadsWell}(x))$

2) $\exists x \exists y (\text{student}(x) \wedge \text{Book}(y) \wedge \text{Likes}(x, y))$

3) $\exists x (\text{student}(x) \wedge \forall y (\text{Book}(y) \rightarrow \text{Likes}(x, y)))$

4) $\forall x (\text{student}(x) \rightarrow \exists y (\text{Book}(y) \wedge \text{Likes}(x, y)))$

5) $\forall x (\text{student}(x) \rightarrow \forall y (\text{Book}(y) \rightarrow \neg \text{Likes}(x, y)))$.

④ Explain the concept of Resolution in AI.

→ Resolution is a rule of inference used for propositional logic and FOL. It is a powerful method for automated theorem proving and plays a critical role in system like logic programming, expert system and AI problem solving.

2) To determine whether a given set of statements is logically consistent.

3) It proves the statement by contradiction.

4) statements are converted into Conjunctive Normal Form, which is conjunction of disjunction

5) If two clauses contain complementary literals, they can be resolved by eliminating those literals and combining the remaining parts of the clauses.

6) General form:
$$\frac{(A \vee P), (\neg A \vee Q)}{(P \vee Q)}$$

Example : Prove that $A \rightarrow C$ given the premises

1) $A \vee B$

2) $\neg B \vee C$

Convert $A \rightarrow C$ to its negation in CNF

$$A \rightarrow C = \neg A \vee C$$

$$\neg(\neg A \vee C) = A \wedge \neg C$$

Initial set of clauses.

$$A \vee B$$

$$\neg B \vee C$$

$$A$$

$$\neg C$$

Apply resolution :

Resolve $A \vee B$ with A : Result is B

Resolve B with $\neg B \vee C$: Result is C

Resolve C with $\neg C$: Result is an empty clause.

$A \rightarrow C$ is true.

⑤ Explain graph plan algorithm.

→ Graph Plan is a planning algorithm used in AI to generate a sequence of actions that achieves a specified goal from a given initial state.

2) A layered structure with alternating levels of state literals and actions.

3) It has two phases:

Construction phase: Build the planning graph to include all possible actions and states up to a level where goal is achievable.

Solution Extraction Phase: Search the graph for a valid plan that satisfies the goal.

4) Avoids generating a large search space like state space planning.

Algorithm:

function GRAPH PLAN (problem) returns solution or failure.

graph $\leftarrow$ Initial-planning-graph (problem)

goals $\leftarrow$ conjuncts (problem. goal)

nogoods $\leftarrow$ an empty hash table.

for $tl = 0$ to ∞ do

if goals all non-mutual in S_t of graph then

 solution $\leftarrow$ EXTRACT-SOLUTION (graph, goals, NUMLevels (graph), nogoods).

if solution $\neq$ failure then return solution

if graph and ~~no~~ nogoods have both leveled off then return failure

graph $\leftarrow$ Expand-graph (graph, problem).

⑥ How state space search is done in AI discuss.

→ State space search in AI is a method used to solve problems by representing them as a set of states and transition. This approach systematically explores the possible states of a problem to find a solution.

Steps in state space:

- 1) Define the problem: specify initial state, goal state, operators and constraints.
- 2) Choose a ~~stage~~ strategy: select an appropriate method eg BFS, DFS, A* etc. base on problem requirement
- 3) Search Execution: uses the strategy to explore the state space systematically. Avoid revisiting already explored states to optimize performance.
- 4) Goal Test: Check if the current state matches the goal.
- 5) Solution Extraction: Trace back the path from the goal state to initial state.

Types of state space search

- 1) Breadth first search
- 2) Depth first search
- 3) Uniform cost search
- 4) A*
- 5) Greedy Search

Applications:

- 1) Pathfinding
- 2) Puzzle solving
- 3) Automated reasoning
- 4) Game playing.

④ Explain the two approaches to searching for a plan in detail.

→ 1) Forward State-Space Search (Progression planning)

In forward state space search, the search starts from initial state and moves step by step towards the goal state by applying applicable actions.

Working

- 1) Initial state: The search begins at the initial state
- 2) Action Selection: At each step, applicable actions are applied to generate successor states.
- 3) Successor states: Each action leads to a new state, representing the result of applying that application
- 4) Goal test: This process continues until a state is generated that satisfies the goal condition.

Eg: A In a robot navigation problem, the robot starts from initial state and applies movements until it reaches goal.

2) Backward State-Space Search (Regression planning)

In backward state space search, the search starts from the goal state and works backward towards initial state by determining what actions could lead to goal.

Working:

- 1) Goal state: The search begins by considering the goal state
- 2) Action Selection: Actions are chosen that could produce the current state
- 3) Precondition Satisfaction: For each action selected, the precondition of the action defines the new sub-goal state.
- 4) Initial state Match: The process continues until a state is generated that matches the initial state.

Example: In a blocks world problem, if the goal is to stack block A on block B the search begins by identifying actions that can produce this state, such as planning A on B, and works backward to find the sequence of action starting from the initial arrangement.